

HW04 Main Report

A. Assignment / Project Overview

This HW04 submission automates three web features from the EShop SUT:

- FR-04 Personal profile management
- FR-08 Checkout
- FR-18 Order management (admin)

The automation uses Playwright with data-driven tests. The repository contains:

- `e2e-tests/profile.spec.ts`
- `e2e-tests/checkout.spec.ts`
- `e2e-tests/admin-orders.spec.ts`
- `e2e-tests/test-data/profile.json`
- `e2e-tests/test-data/checkout.json`
- `e2e-tests/test-data/admin-orders.json`

Playwright configuration is present in:

- `playwright.config.js`
- `playwright.config.ts`

Relevant run commands from `package.json`:

- `npm run test:e2e`
- `npm run test:e2e:chromium`
- `npm run test:e2e:firefox`
- `npm run test:e2e:webkit`

B. Automation Report

Automated features

Feature	Spec file	Data file
FR-04 Personal profile management	<code>e2e-tests/profile.spec.ts</code>	<code>e2e-tests/test-data/profile.json</code>
FR-08 Checkout	<code>e2e-tests/checkout.spec.ts</code>	<code>e2e-tests/test-data/checkout.json</code>
FR-18 Order management (admin)	<code>e2e-tests/admin-orders.spec.ts</code>	<code>e2e-tests/test-data/admin-orders.json</code>

Test cases automated

Feature	Automated cases
FR-04 Personal profile management	15
FR-08 Checkout	14

Feature	Automated cases
FR-18 Order management (admin)	12
Total	41

Execution evidence

The authoritative Playwright HTML report is [playwright-report/index.html](#). The current execution evidence shows:

- 41 automated test cases
- 126 executed test runs
- 109 passed
- 17 failed
- 0 skipped

Browser/project results from the report evidence:

- Chromium: 42 executed, 39 passed, 3 failed
- Firefox: 42 executed, 39 passed, 3 failed
- WebKit: 42 executed, 31 passed, 11 failed

The browser projects configured in [playwright.config.js](#) and [playwright.config.ts](#) are:

- Chromium
- Firefox
- WebKit

Test data and assertion patterns

Test data files:

- [e2e-tests/test-data/profile.json](#)
- [e2e-tests/test-data/checkout.json](#)
- [e2e-tests/test-data/admin-orders.json](#)

Assertion patterns visible in the tests:

- [toBeVisible\(\)](#)
- [toHaveURL\(\)](#)
- [toHaveValue\(\)](#)
- [toBeDisabled\(\)](#)
- [toBeEditable\(\)](#)
- [not.toHaveValue\(\)](#)
- [toContain\(\)](#)

Browser/project coverage

The repository configuration covers 3 browsers and 3 browser projects in the Playwright report:

- Chromium
- Firefox
- WebKit

C. AI-Generated Script Review

The three spec files look like AI-assisted first drafts that were then adapted into a working suite structure: each feature is organized with helper functions, data files, loops over test data, and multiple assertion styles. That is a good starting point, but the execution evidence shows several assumptions that a human reviewer still needs to challenge.

FR-04 Personal profile management

What AI did well:

- It produced a data-driven profile suite with separate test data in `profile.json`.
- It covered more than the minimum number of cases: valid updates, invalid phone inputs, order history, and field-state checks.
- It reused helpers for registration/login, order seeding, and locating profile inputs, which keeps the test body readable.

Gaps and problems:

- The valid profile-update tests rely on `page.once('dialog', ...)` and then immediately assert `dialogMessage`. The current evidence shows the message stays empty in the two valid-profile failures for both valid-profile names.
- The shipping-address case is not an application bug by itself. The evidence shows the test submitted a phone value that the page rejected, so the failure is a test-data/setup issue rather than a proven defect in the profile save flow.
- The same suite mixes backend setup with UI actions, but the failure evidence shows the UI state after update is not being verified in a stable way.
- The selectors are functional, but several are fragile because they depend on translated label text and `xpath=following-sibling::...` chains instead of a more stable locator strategy.

Human review/correction needed:

- Replace the immediate dialog-string assertion with a wait for the actual confirmation behavior, or verify the persisted profile state after save.
- Treat the shipping-address failure as a data/setup correction, not as proof that the application is broken.

FR-08 Checkout

What AI did well:

- It created a reusable checkout flow with a login helper, cart navigation helper, and a small data set for coupon scenarios.
- It exercises positive and negative coupon cases and checks several checkout UI states.
- It uses data-driven iteration over `checkout.json`, which is aligned with the assignment requirement.

Gaps and problems:

- `goToCheckout()` assumes that clicking the checkout button will always land on `/checkout`, but the failure evidence shows the page remains on `/login`.
- Because the suite starts from a login helper, the tests appear to expect authentication to persist. The report indicates that assumption is not holding during execution.
- Several checkout failures stop before reaching the intended assertions because the navigation never completes. That makes those failures validation problems in the test flow, not confirmed product bugs.

Human review/correction needed:

- Check the login/session setup and wait for the actual authenticated checkout state before asserting route changes.
- The current failure pattern suggests the test is validating the wrong precondition first, so the checkout assertions are not yet trustworthy as feature verification.

FR-18 Order management (admin)

What AI did well:

- It produced a more complete admin suite with login, order seeding, state transitions, and visibility checks.
- It uses request-based setup to create orders in known states, which is a sensible pattern for admin workflow testing.
- It includes several coverage points for order rows and buttons, not just a single happy path.

Gaps and problems:

- No failure evidence is present for this suite in the current report bundle, so the repository does not prove that all admin scenarios are unstable.
- The quality of the admin suite is still only partially verifiable because the current report evidence does not show a completed pass/fail matrix for each test.
- Some assertions check table text and button visibility, which is useful, but they do not always prove a state transition occurred unless combined with stronger state verification.

Human review/correction needed:

- Confirm whether the admin transition assertions are sufficient to prove the state machine behavior, or whether extra state checks are needed after each action.
- Because there are no visible failures here, this feature is the strongest of the three, but the absence of failures is not proof that the tests are perfect.

Gap Analysis Table

Area	AI-generated approach	Evidence/problem	Impact	Human review/correction	Status
Selectors	Relies on translated labels and <code>xpath=following-sibling:... lookups</code> in profile and checkout helpers	Fragile locator strategy visible in <code>profile.spec.ts</code> and <code>checkout.spec.ts</code>	Higher maintenance risk and possible selector breakage when the UI changes	Prefer role-, label-, or test-id-based locators where possible	Partial
Authentication	Uses helper-based login and assumes the session survives through checkout and profile flows	Checkout failures stay on <code>/login</code> after checkout click	Checkout assertions become inconclusive because the intended page is never reached	Verify authenticated state before asserting navigation, and wait for the real post-login route	Gap

Area	AI-generated approach	Evidence/problem	Impact	Human review/correction	Status
Test data	Uses separate JSON files for profile, checkout, and admin cases	Profile shipping-address failure uses a phone value that is rejected by the UI	The save test fails for data/setup reasons rather than proving the feature	Align test inputs with validation rules before asserting success	Partial
Assertions	Uses dialog-message equality checks and text-visibility checks	Profile success assertions capture an empty dialog message in the report	Weak synchronization makes the test fail before validating the intended behavior	Assert the actual UI confirmation or persisted state instead of a fragile captured string	Gap
Synchronization	Uses immediate assertions after click actions	Visible profile and checkout failures fail at the assertion boundary	Tests can fail before reaching the intended feature behavior	Add waits for the real application state change or confirmation element	Gap
Navigation/state	Assumes checkout flow reaches <code>/checkout</code> after clicking the payment button	Report shows <code>/login</code> instead of <code>/checkout</code> for checkout failures	The tests do not validate checkout behavior reliably	Check session/state setup and wait for the correct route transition	Gap
Data-driven testing	Uses loops over JSON data to generate multiple cases	Present and working in all three suites	Good coverage and readability	Keep the pattern, but pair it with stronger state assertions	Complete
Browser coverage	Configured for Chromium, Firefox, and WebKit in both Playwright configs	Report artifacts show failures across multiple browsers for profile and checkout	Cross-browser support exists, but reliability still varies by feature	Re-run and verify each feature-browser combination after fixing the setup issues	Complete
Error handling	Minimal explicit error handling; tests rely on direct Playwright assertions	Failures surface as assertion errors rather than clear feature-state checks	Harder to tell whether the feature or the test failed first	Add explicit precondition checks and clearer wait conditions	Partial

Overall assessment:

- The AI-generated scripts provide useful initial coverage and are clearly better than a single generic one-test-per-feature draft.

- They are not trustworthy without human review.
- The most important AI errors are the fragile dialog assertion in FR-04, the incorrect navigation assumption in FR-08, and the data/setup mismatch in the shipping-address case.
- The current failures mean parts of FR-04 and FR-08 are still not reliably validated end to end.
- Human review is necessary because the current suite structure is reasonable, but the execution evidence shows that several tests fail before proving the intended requirement.

D. Test Results

Metric	Result
Features	3
Test cases automated	41
Test cases executed	126
Passed	109
Failed	17
Skipped	0
Browser runs	3
Bugs	0 confirmed application bugs

E. Bugs and Failures

The current evidence does not confirm an application defect.

Observed failures:

- FR-04 valid profile update assertions fail because the expected dialog message is empty.
- FR-04 shipping-address save fails because the test submits a phone value that the UI rejects.
- FR-08 checkout navigation assertions fail because the flow remains on `/login` instead of reaching `/checkout`.

These failures are best classified as:

- test synchronization / assertion issues
- test setup or application-state issues
- test-data issues
- potentially incorrect assumptions about the UI flow

F. Conclusion

The repository contains a substantial Playwright automation submission for three web features with separate JSON data and HTML report evidence. The strongest parts are the feature coverage, data-driven structure, and browser configuration. The biggest human-review need is in the profile and checkout flows, where the current tests make assumptions about dialogs and navigation that the report evidence does not support.

The submission still needs manual completion for the AI audit appendix, AI critique, demo video link, student ID banner evidence, and commit log.